



Esercitazione 8

Filtering & Denoising



Introduzione

Questa esercitazione affronta tecniche base ma molto utili di **filtering & denoising** per segnali discreti, con applicazioni pratiche su un breve file audio e su una serie storica finanziaria. Partendo da filtri non ricorsivi (media mobile) e passando ai filtri ricorsivi semplici (leaky integrator), l'obiettivo è mostrare come attenuare il rumore preservando le componenti informative del segnale e come applicare lo stesso concetto a stime mobili (media — e volatilità) su dati di prezzo. Nel PDF sono presentati i concetti matematici (risposte all'impulso, ricorsione, forma chiusa del leaky integrator), esempi di codice in Python che caricano i dati (`jingle.mat` , `data.mat`), producono plot nel dominio del tempo e della frequenza, applicano convoluzioni con finestre di diversa ampiezza e calcolano misure come ritorni e volatilità.

Punti chiave

- **Media mobile:** filtro semplice, non ricorsivo; attenua rumore ad alta frequenza ma introduce ritardo e appiattimento delle transizioni.
- **Leaky integrator:** versione ricorsiva che simula una media esponenziale pesata; più efficiente per finestre "lunghe" (parametro λ vicino a 1).
- **Trade-off** tra smoothing e preservazione del dettaglio: finestre più grandi o λ più vicini a 1 aumentano smoothing ma cancellano variazioni rapide.
- **Applicazioni:** denoising audio (analisi tempo/frequenza) e stima dinamica di ritorni/volatilità per dati finanziari.



Riassunto Esercitazione 8

Obiettivo: mostrare e confrontare tecniche semplici di filtraggio (media mobile e leaky integrator) per ridurre rumore in segnali audio e per stimare medi mobili / volatilità su serie storiche finanziarie; evidenziare effetti pratici (ritardi, modifiche spettrali, scelta parametri).

Take-home messages generali

- La media mobile è semplice e intuitiva ma non ricorsiva: buona per smoothing locale ma introduce ritardo e perdita di dettaglio transitorio.
- Il leaky integrator (media esponenziale) ottiene simile smoothing con complessità computazionale molto più bassa e senza dover memorizzare tutta la finestra; il parametro λ governa la "memoria".
- In ambito audio osserva-se attenuazione delle componenti ad alta frequenza nello spettro dopo il filtraggio; in ambito finanziario il window size M o λ definiscono quanto rapidamente le stime reagiscono a shock di mercato.
- Attenzione a dettagli implementativi: modalità di convoluzione (`valid` vs `same`), tipi di dato per salvataggio audio e definizione corretta dei ritorni (vedi commenti sotto).



Filtering and Denoising

Contesto

- Tema principale: **filtraggio per denoising** (riduzione del rumore da un segnale).
- Approccio: tecniche di **media mobile e integratori discreti** (con o senza perdita).

Moving Average (Media Mobile)

- Idea: sostituire ogni campione con la **media locale**.
 - Esempio semplice:

$$y_n = \frac{x_n + x_{n-1}}{2}$$

- Caso generale con finestra di ampiezza M :

$$y_n = \frac{1}{M} \sum_{k=0}^{M-1} x_{n-k}$$

- **Risposta all'impulso** del filtro media mobile:

$$h_n = \begin{cases} \frac{1}{M}, & 0 \leq n < M \\ 0, & \text{altrimenti} \end{cases}$$

Dalla Media Mobile a un Filtro Ricorsivo

- Per finestre M grandi, si passa a una forma ricorsiva.
- **Leaky integrator (integratore con perdita):**

$$y_n = \lambda y_{n-1} + (1 - \lambda)x_n$$

- Ricorsivo perché usa l'uscita precedente.
- $\lambda \approx 1$ simula una media mobile lunga.

Risposta all'impulso del Leaky Integrator

- **Ricorsione:**

$$h_n = \lambda h_{n-1} + (1 - \lambda)\delta_n$$

- Sviluppo:

$$h_0 = (1 - \lambda), \quad h_1 = \lambda(1 - \lambda), \quad h_2 = \lambda^2(1 - \lambda), \dots$$

- Forma chiusa:

$$h_n = \lambda^n(1 - \lambda)u_n$$

Integratore e Versione "Leaky"

- **Integratore discreto standard:**

$$y_n = y_{n-1} + x_n$$

È un **accumulatore illimitato**.

- **Leaky integrator** introduce $\lambda < 1$ per evitare esplosione:
 - Mantiene solo una frazione λ della memoria accumulata.
 - Aggiunge una frazione $(1 - \lambda)$ del nuovo valore.

✓ Take-home messages:

- Il **filtro media mobile** è un metodo semplice di denoising, ma non ricorsivo.
- Si può trasformarlo in una versione **ricorsiva** con il **leaky integrator**.
- Il parametro λ controlla quanta memoria del passato viene mantenuta e quanta viene "dimenticata".
- L'integratore puro tende a divergere, mentre quello con perdita è stabile e adatto al denoising.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
from scipy import fftpack as f
import scipy.io.wavfile as wav
```

🚀 Esercizio 1: Denoising di un segnale audio

(a) Leggere il file `jingle.mat` (scaricabile dalla cartella del corso) utilizzando il comando:

```
np.squeeze(sio.loadmat('jingle.mat')['jingle'])
```

Il file contiene un breve segnale audio $x(t)$. Fare un plot del segnale (nel tempo e in frequenza). Salvare il segnale $x(t)$ come file audio utilizzando `sio.wavfile.write('jingle.wav', 44000, x)` e provare ad ascoltarlo.

(b) Creare un vettore $n(t)$ della stessa lunghezza del segnale contenente rumore gaussiano con media $\mu = 0$ e deviazione standard $\sigma = 0.01$. Utilizzare il comando `np.random.normal(mu, sigma, N)` per ottenere N campioni.

(c) Fare un plot del segnale rumoroso $x_n(t) = x(t) + n(t)$ (nel tempo e in frequenza), salvare il segnale $x_n(t)$ come file audio e provare ad ascoltarlo.

(d) Calcolare la media locale (filtro a media mobile) definita come:

$$\tilde{x}(t) = \frac{1}{M} \sum_{k=0}^{M-1} x_n(t - k),$$

considerare $M = 10$. Calcolare $\tilde{x}(t)$ utilizzando l'operazione di convoluzione. Salvare i segnali ottenuti come file audio e provare ad ascoltarli.

(e) Calcolare la media locale pesata esponenzialmente utilizzando il leaky integrator:

$$h(t) = (1 - \lambda)\lambda^t u(t),$$

dove $u(t)$ è la funzione gradino, t è un vettore di 100 punti equispaziati compresi tra 0 e 99 e $\lambda = 0.9$. Calcolare $\tilde{x}(t)$ utilizzando la convoluzione con il filtro $h(t)$. Salvare il segnale ottenuto come file audio e provare ad ascoltarlo.

```
In [ ]: # (a) Segnale originale
x = np.squeeze(sio.loadmat('jingle.mat')['jingle'])
print(f'Segnale originale caricato: {x.shape[0]} campioni')

# Plot nel dominio del tempo
plt.figure(figsize=(10, 4))
plt.plot(x)
plt.xlabel('Samples')
plt.ylabel('Ampiezza')
plt.title('Segnale originale - Dominio del tempo')
plt.grid(True)
plt.show()

# Spettro
X = f.fft(x)
plt.figure(figsize=(10, 4))
plt.plot(np.arange(len(X)) / len(X), np.abs(X))
plt.xlabel('Frequenza normalizzata')
plt.ylabel('|X|')
plt.title('Spettro del segnale originale')
plt.grid(True)
plt.show()

# Salvataggio audio
sio.wavfile.write('jingle.wav', 44000, x)
print("File 'jingle.wav' salvato.")
```

```

# (b) Rumore gaussiano
n = np.random.normal(0, 0.01, len(x))
xn = x + n
print("Rumore gaussiano aggiunto ( $\sigma = 0.01$ )")

# Plot dominio del tempo
plt.figure(figsize=(10, 4))
plt.plot(xn)
plt.xlabel('Samples')
plt.ylabel('Ampiezza')
plt.title('Segnale con rumore - Dominio del tempo')
plt.grid(True)
plt.show()

# Spettro
Xn = f.fft(xn)
plt.figure(figsize=(10, 4))
plt.plot(np.abs(Xn))
plt.xlabel('Indice di frequenza')
plt.ylabel('|X_noisy|')
plt.title('Spettro del segnale rumoroso')
plt.grid(True)
plt.show()

# Salvataggio audio
sio.wavfile.write('jingle_noisy.wav', 44000, xn)
print("File 'jingle_noisy.wav' salvato.")

# (d) Filtro a media mobile
M = 10
h = np.ones(M) / M
y_ma = np.convolve(xn, h, mode='valid')
print(f"Filtro a media mobile applicato (M = {M})")

# Plot tempo
plt.figure(figsize=(10, 4))
plt.plot(y_ma)
plt.xlabel('Samples')
plt.ylabel('Ampiezza')
plt.title('Segnale filtrato (Media Mobile) - Dominio del tempo')
plt.grid(True)
plt.show()

# Spettro
Y_ma = f.fft(y_ma)
plt.figure(figsize=(10, 4))
plt.plot(np.abs(Y_ma))
plt.xlabel('Indice di frequenza')
plt.ylabel('|X_denoised|')
plt.title('Spettro del segnale filtrato (Media Mobile)')
plt.grid(True)
plt.show()

# Salvataggio audio
sio.wavfile.write('jingle_denoised.wav', 44000, y_ma)
print("File 'jingle_denoised.wav' salvato.")

# (e) Filtro Leaky Integrator
lmbd = 0.9
h_li = (1 - lmbd) * np.power(lmbd, np.arange(100))
y_li = np.convolve(xn, h_li, mode='valid')
print(f"Leaky integrator applicato ( $\lambda = \{lmbd\}$ )")

# Plot tempo
plt.figure(figsize=(10, 4))
plt.plot(y_li)
plt.xlabel('Samples')
plt.ylabel('Ampiezza')
plt.title('Segnale filtrato (Leaky Integrator) - Dominio del tempo')
plt.grid(True)
plt.show()

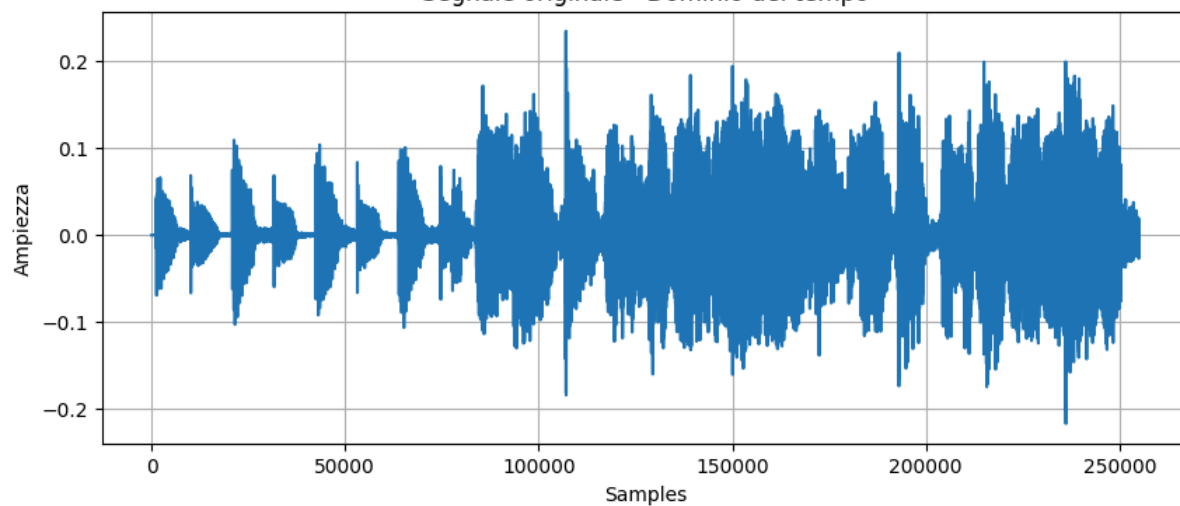
# Spettro
Y_li = f.fft(y_li)
plt.figure(figsize=(10, 4))
plt.plot(np.abs(Y_li))
plt.xlabel('Indice di frequenza')
plt.ylabel('|X_denoised_li|')
plt.title('Spettro del segnale filtrato (Leaky Integrator)')
plt.grid(True)
plt.show()

# Salvataggio audio
sio.wavfile.write('jingle_denoised_li.wav', 44000, y_li)
print("File 'jingle_denoised_li.wav' salvato.")

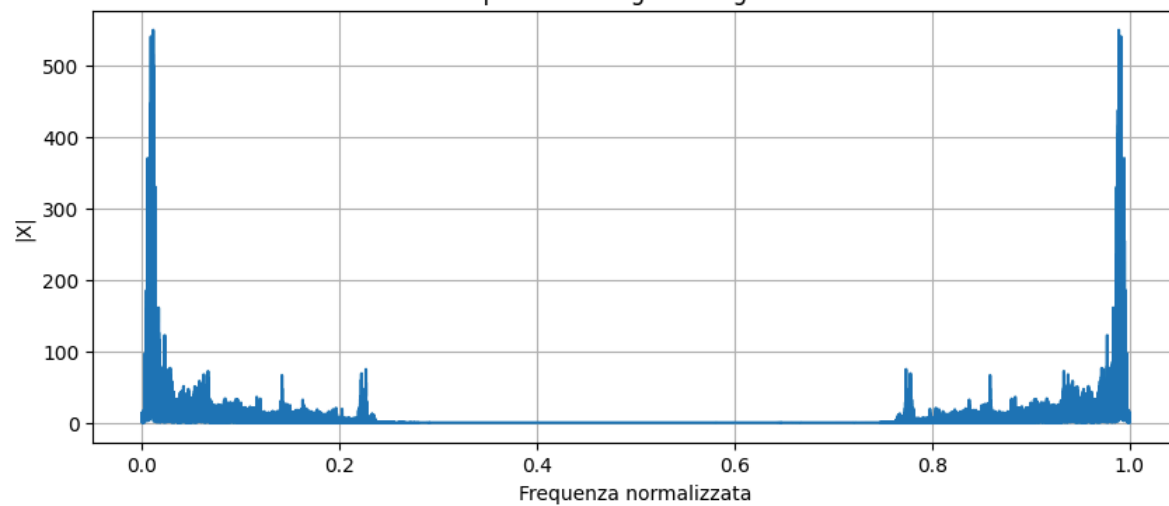
```

Segnale originale caricato: 255000 campioni

Segnale originale - Dominio del tempo



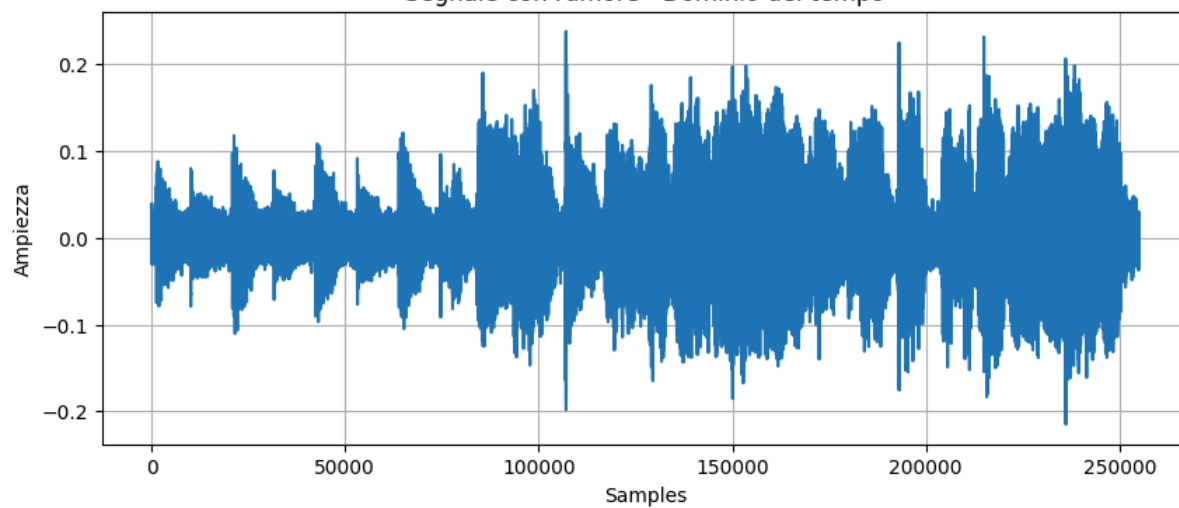
Spettro del segnale originale



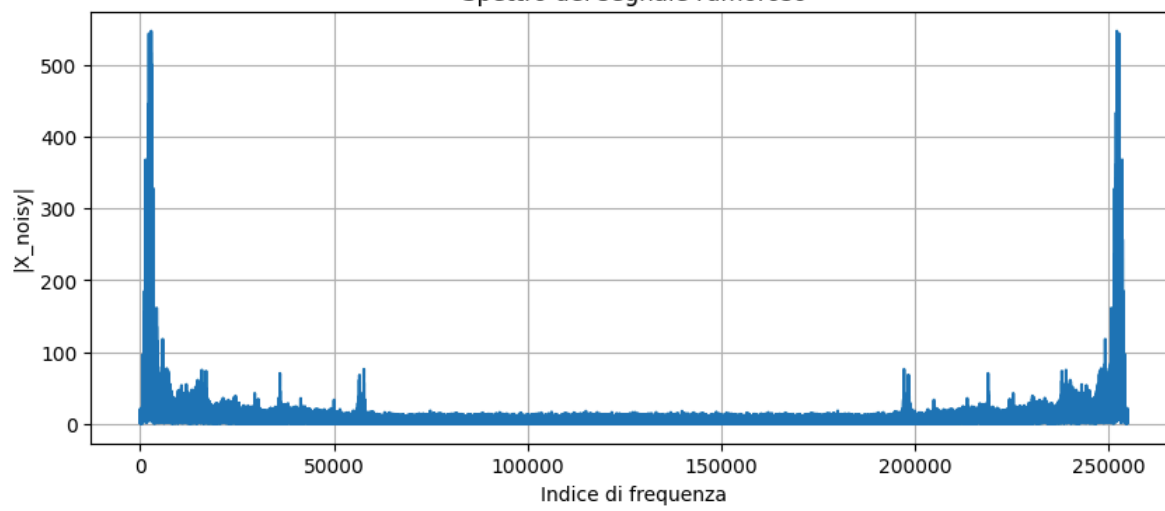
File 'jingle.wav' salvato.

Rumore gaussiano aggiunto ($\sigma = 0.01$)

Segnale con rumore - Dominio del tempo



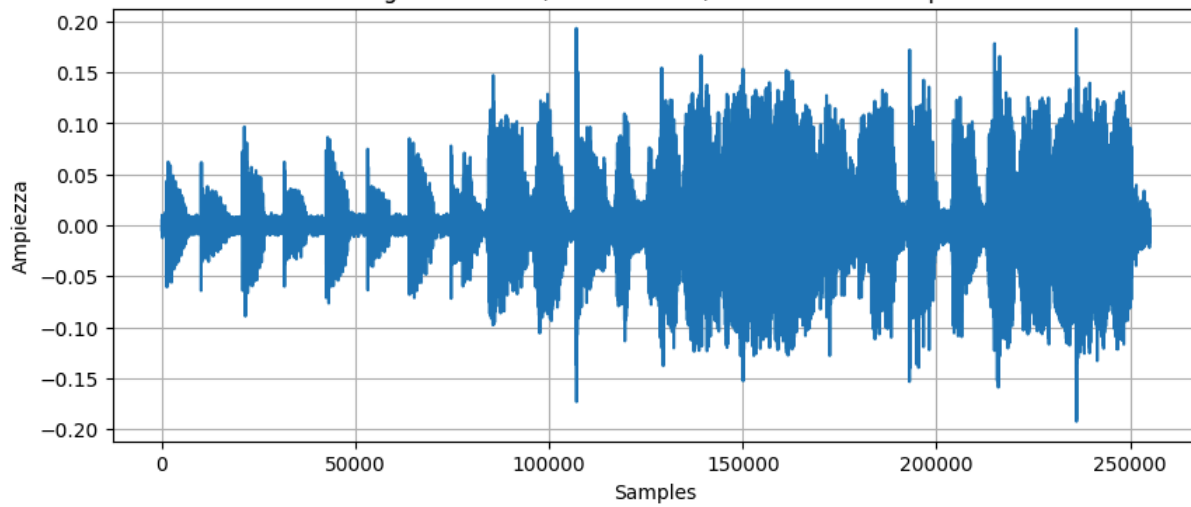
Spettro del segnale rumoroso



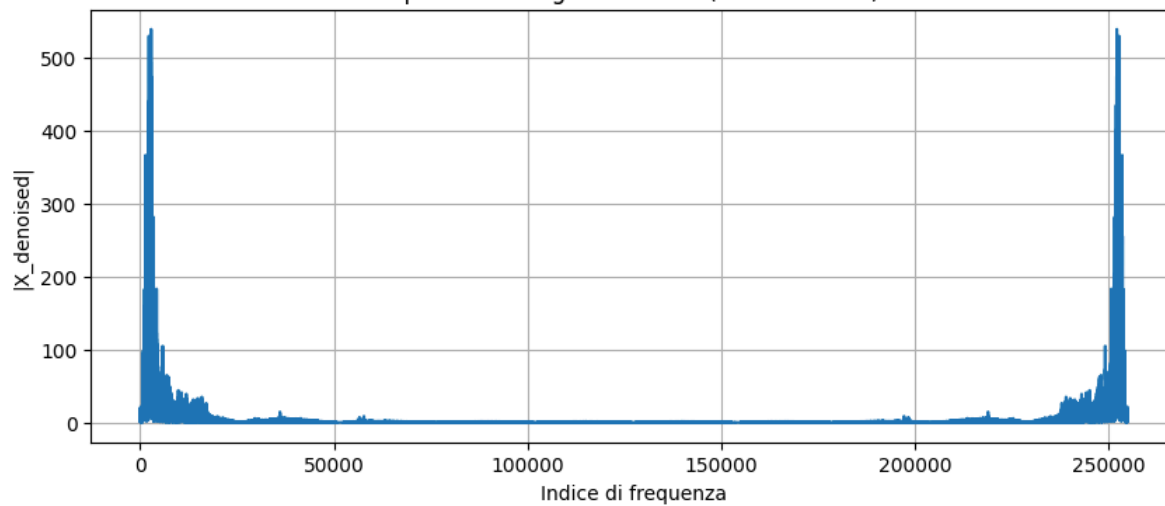
File 'jingle_noisy.wav' salvato.

Filtro a media mobile applicato ($M = 10$)

Segnale filtrato (Media Mobile) - Dominio del tempo

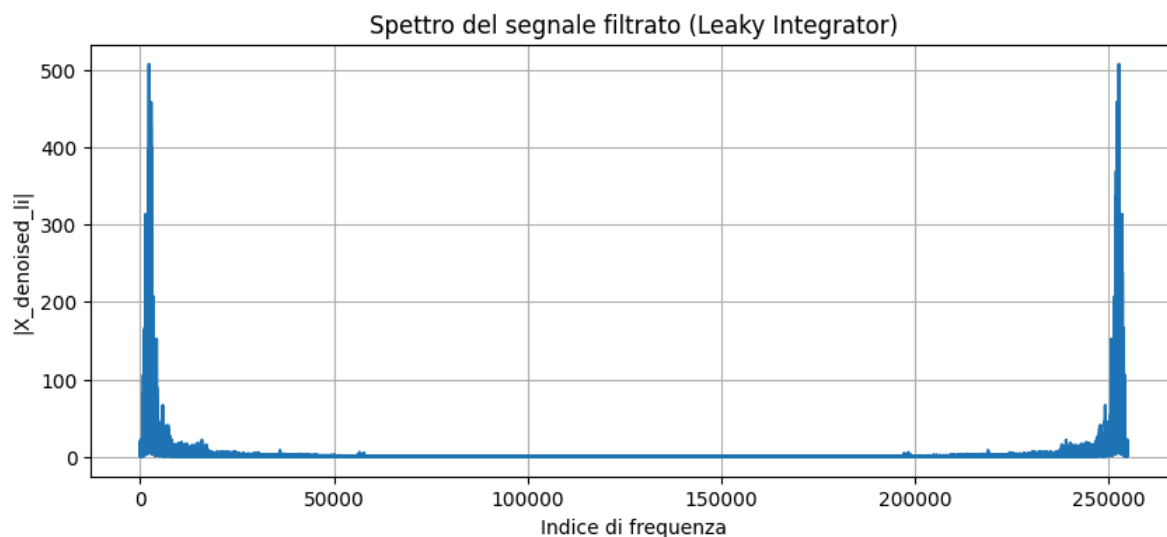
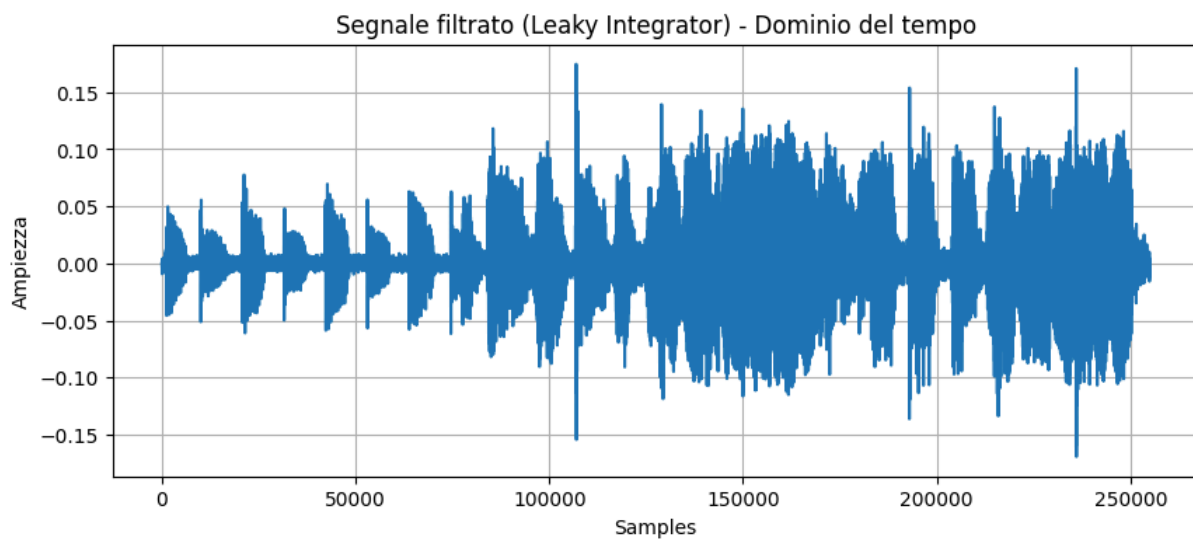


Spettro del segnale filtrato (Media Mobile)



File 'jingle_denoised.wav' salvato.

Leaky integrator applicato ($\lambda = 0.9$)



File 'jingle_denoised_li.wav' salvato.

COMMENTO.

Obiettivi

- Caricare `jingle.mat`, visualizzare segnale in dominio del tempo e della frequenza, salvare come `.wav`.
- Aggiungere rumore gaussiano, visualizzare e salvare.
- Applicare filtro a **media mobile** ($M = 10$) e **leaky integrator** ($\lambda = 0.9$) via convoluzione e confrontare risultati (tempo + spettro), salvare audio denoised.

Commento sui plot

- **Segnale originale — tempo:** forma d'onda con contenuto audio riconoscibile; assenza di alta variabilità dovuta al rumore.
- **Spettro originale:** picchi a frequenze fondamentali del jingle; spettro non piatto.
- **Segnale rumoroso (x_n):** nel plot temporale vedrai l'onda originale "sommersa" da fluttuazioni casuali; nello spettro si nota aumento dell'energia su bande più alte (rumore bianco sparge energia).
- **Segnale filtrato (media mobile $M=10$):** nel dominio del tempo si osserva una versione più liscia del segnale; transienti rapidi attenuati; possibili artefatti di ritardo e perdita di dettaglio. Nello spettro vedrai riduzione dell'energia alle frequenze più alte (low-pass effect).
- **Segnale filtrato (leaky integrator $\lambda=0.9$):** effetto simile alla media mobile ma con decadimento esponenziale della memoria; probabilmente una risposta meno "scalettata" rispetto alla media mobile e con comportamento più graduale. Anche qui lo spettro mostra attenuazione delle alte frequenze.
- **Nota pratica sui plot nel codice del PDF:** si usa `mode='valid'` nella `np.convolve` — questo **accorcia** il vettore output. Se vuoi mantenere la lunghezza originale e confrontare shape/overlay, considera `mode='same'` o gestire esplicitamente il padding.

Take-home message

- Media mobile e leaky integrator riducono efficacemente il rumore ad alta frequenza ma **distorcono** transitori e introducono ritardo; scelta di M o λ è un compromesso tra smoothing e conservazione del dettaglio.
- Per audio, normalizzazione e tipo di dato per il salvataggio sono importanti per non introdurre artefatti.

✦ Esercizio 2: Misurare il rischio finanziario

(a) Leggere il file `data.mat` (scaricabile dalla cartella del corso). Il file contiene due matrici `dates_ts` e `price_ts` che descrivono l'evoluzione del prezzo di un pacchetto azionario in 5 anni (campionato con frequenza giornaliera). Fare un plot dei dati.

(b) Calcolare il ritorno dell'investimento $r(t)$ definito come:

$$r(t) = \frac{p(t) - p(t-1)}{p(t-1)},$$

dove $p(t)$ è il prezzo delle azioni al tempo t . Calcolare il ritorno per i dati forniti e fare un plot del risultato.

(c) Stimare la volatilità utilizzando la finestra mobile di ampiezza M . Calcolare il ritorno medio $\bar{r}(t)$ sugli ultimi M campioni:

$$\bar{r}(t) = \frac{1}{M} \sum_{k=0}^{M-1} r(t-k),$$

e la volatilità:

$$\sigma(t) = \sqrt{\frac{1}{M} \sum_{k=0}^{M-1} (r(t-k) - \bar{r}(t))^2}.$$

Considerare diversi valori di $M = 20, 50, 70, 100$ e per ognuno calcolare $\bar{r}(t)$ e $\sigma(t)$ utilizzando l'operazione di convoluzione e fare i plot dei risultati.

(d) Calcolare il ritorno medio e la volatilità utilizzando il leaky integrator presentato nell'esercizio precedente (utilizzando $\lambda = 0.94$) e fare un plot del risultato.

```
In [ ]: # (a) Lettura dati
dates = np.squeeze(sio.loadmat('data.mat')['dates_ts'])
price = np.squeeze(sio.loadmat('data.mat')['price_ts'])

print(f"Dati caricati: {len(price)} prezzi giornalieri.")

# Plot andamento prezzi
plt.figure(figsize=(10, 4))
plt.plot(np.arange(len(dates)), price)
plt.xlabel('Giorni')
plt.ylabel('Prezzo')
plt.title('Andamento del prezzo')
plt.grid(True)
plt.show()

# (b) Calcolo ritorno giornaliero
return_ts = (price[1:] - price[:-1]) / price[1:]
print(f"Ritorni calcolati: {len(return_ts)} campioni.")

plt.figure(figsize=(10, 4))
plt.plot(return_ts, '.', markersize=2)
plt.xlabel('Giorni (dal 2° in poi)')
plt.ylabel('Ritorno')
plt.title('Ritorno giornaliero')
plt.grid(True)
plt.show()

# (c) Media mobile e volatilità per vari M
M_vec = [20, 50, 70, 100]
fig, axs = plt.subplots(2, 1, figsize=(12, 6), sharex=True)

for M in M_vec:
    h = np.ones(M) / M
    return_avg = np.convolve(return_ts, h, mode='valid')
    volatility = np.sqrt(np.convolve(return_ts**2, h, mode='valid') - return_avg**2)

    axs[0].plot(return_avg, label=f'M={M}')
    axs[1].plot(volatility, label=f'M={M}')

axs[0].set_ylabel('Ritorno medio')
axs[0].set_title('Ritorno medio con finestra mobile')
axs[0].grid(True)
axs[0].legend()

axs[1].set_xlabel('Giorni')
axs[1].set_ylabel('Volatilità')
axs[1].set_title('Volatilità con finestra mobile')
axs[1].grid(True)
axs[1].legend()

plt.tight_layout()
plt.show()

# (d) Leaky integrator (λ=0.94)
lmbd = 0.94
h_li = (1 - lmbd) * np.power(lmbd, np.arange(100))

return_avg_li = np.convolve(return_ts, h_li, mode='valid')
volatility_li = np.sqrt(np.convolve(return_ts**2, h_li, mode='valid') - return_avg_li**2)

plt.figure(figsize=(12, 5))
plt.subplot(2, 1, 1)
plt.plot(return_avg_li)
plt.ylabel('Ritorno medio')
plt.title(f'Ritorno medio (Leaky Integrator, λ={lmbd})')
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(volatility_li)
plt.xlabel('Giorni')
plt.ylabel('Volatilità')
plt.title(f'Volatilità (Leaky Integrator, λ={lmbd})')
plt.grid(True)

plt.tight_layout()
plt.show()

# (Extra) Volatilità pesata esponenzialmente (iterativa)
exp_average = 0
exp_volatility = np.zeros(len(return_ts) + 1)
```

```

for i in range(len(return_ts)):
    exp_average = lmbd * exp_average + (1 - lmbd) * return_ts[i]
    exp_volatility[i + 1] = lmbd * exp_volatility[i] + (1 - lmbd) * (return_ts[i] - exp_average)**2

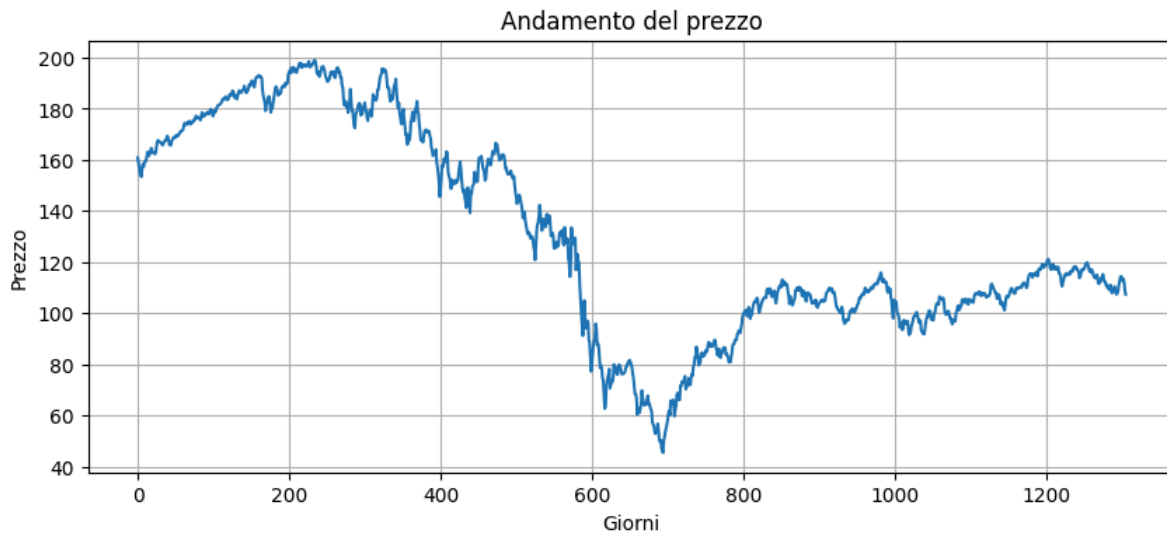
exp_volatility = np.sqrt(exp_volatility[1:])

plt.figure(figsize=(10, 4))
plt.plot(exp_volatility)
plt.xlabel('Giorni')
plt.ylabel('Volatilità')
plt.title(f'Volatilità (Pesata Esponenzialmente,  $\lambda={lmbd}$ )')
plt.grid(True)
plt.show()

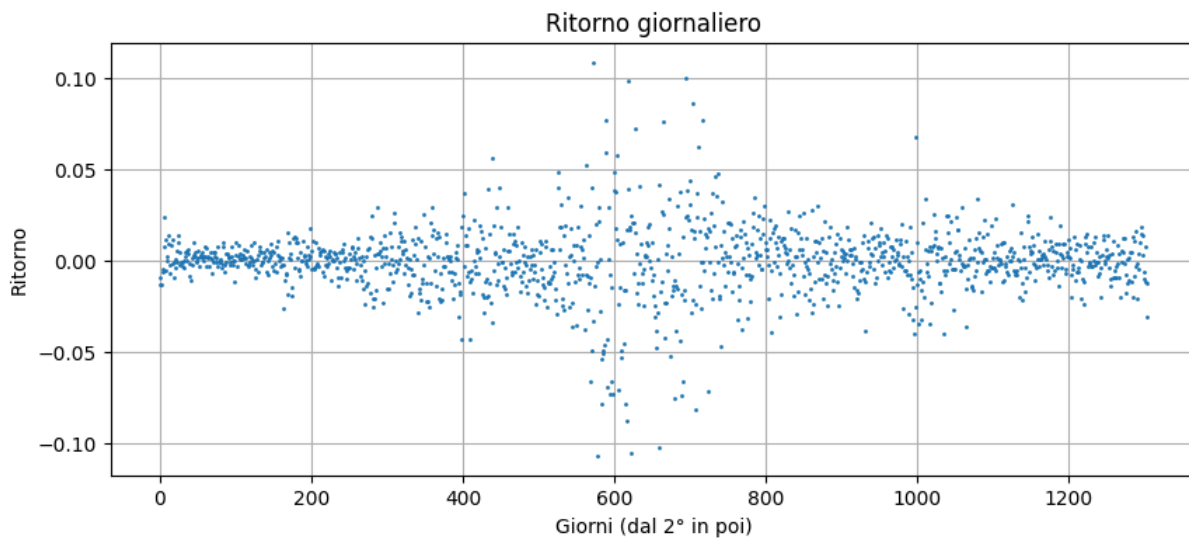
print("Calcoli completati per tutti i punti (a)-(d).")

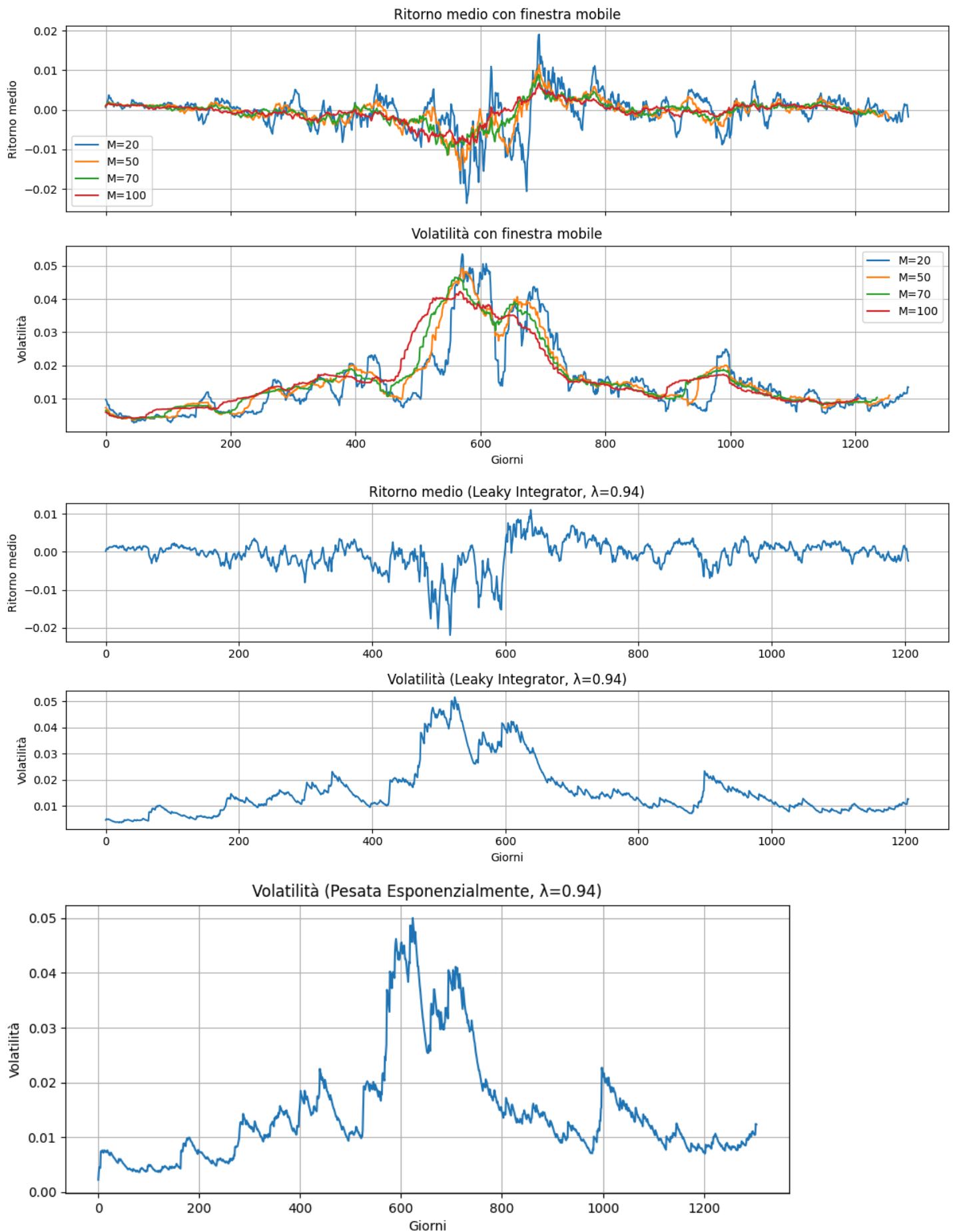
```

Dati caricati: 1306 prezzi giornalieri.



Ritorni calcolati: 1305 campioni.





Calcoli completati per tutti i punti (a)-(d).

COMMENTO.

Obiettivi

- Caricare `data.mat` (`dates_ts`, `price_ts`), plottare prezzi.
- Calcolare i ritorni e plottarli.
- Stimare ritorno medio e volatilità con media mobile per diversi M (20,50,70,100).
- Ripetere usando leaky integrator ($\lambda = 0.94$) e mostrare volatilità esponenzialmente pesata.

Commento sui plot

- **Prezzo (time series):** tipica traiettoria di mercato su 5 anni; controlla trend, stagionalità o eventi anomali (salti).

- **Ritorni giornalieri (scatter/points):** distribuzione centrata attorno a zero; osserva outlier (giorni ad alta variazione).
- **Ritorno medio con finestre M diverse:** curve più "smussate" per M grandi; finestre piccole seguono più da vicino i picchi. Confronta le curve per capire sensibilità alla scelta di M.
- **Volatilità (finestra mobile):** per M piccoli vedi più rumore nella stima della volatilità; per M grandi la volatilità è più stabile ma reagisce lentamente agli shock.
- **Leaky integrator ($\lambda=0.94$):** volatilità pesata esponenzialmente risponde più rapidamente degli M grandi ma meno rumorosa rispetto a finestre molto piccole; utile per dare più peso agli eventi recenti.
- **Grafico dell' exp_volatility (implementazione iterativa):** attento all'inizializzazione — i primi punti possono essere instabili (la versione iterativa del PDF inizializza a zero).

Take-home message

- La stima di ritorno medio e volatilità dipende fortemente dalla finestra scelta: finestre più grandi stabilizzano le stime ma ritardano la risposta; EWMA/leaky integrator danno un buon compromesso con sensibilità ai dati recenti.